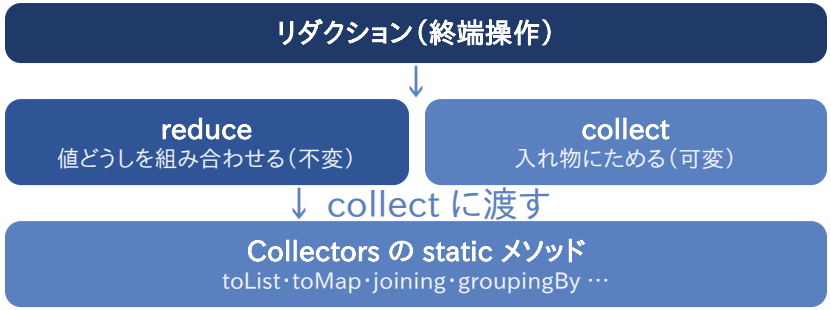
Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 3 Java ストリーム API　#07

リダクションとCollectors

reduce・collect・toMap・joining・groupingBy・partitioningBy と下流のCollectors

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java SE 17 API の java.util.stream パッケージの説明・Stream・Collectors・IntSummaryStatistics、The Java Tutorials の Reduction）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21）で実行して確かめた

全体像 要素を 1 つの結果にまとめる



書き方	戻り値	0 個のとき
reduce(op)	Optional<T>	empty
reduce(id, op)	T	id
reduce(id, acc, comb)	U	id
collect(Collector)	Collector 次第	空の入れ物
collect(sup, acc, comb)	R	sup が作ったもの

> Collector（インタフェース）と Collectors（ユーティリティクラス）を区別。

出典：API java.util.stream パッケージの説明「Reduction operations」「Mutable reduction」、Stream.reduce・collect

第1章

reduce と collect

単位元・アキュムレータ・コンバイナ。

reduce の 3 つの形

```
Stream.of(1,2,3).reduce(10, (x, y) -> x + y)  // 16（10 も足す）
Stream.of(1,2,3).reduce((x, y) -> x + y)      // Optional[6]
Stream.<Integer>empty().reduce((x, y) -> x + y) // Optional.empty
Stream.<Integer>empty().reduce(0, Integer::sum)  // 0

Integer o = Stream.of(1,2,3).reduce((x, y) -> x + y);
// × Optional<Integer> cannot be converted to Integer
```

3 引数：要素と結果の型が違う

```
Stream.of("a","bb","ccc")
  .reduce(0, (n, s) -> n + s.length(), Integer::sum)  // 6
```



単位元でない値（10）を渡すと、並列では何回も足されて結果が変わる（#09）。

出典：API Stream.reduce（3 つの版）／JDK 17 でのコンパイルと実行の結果

collect の 3 引数版

```
StringBuilder sb = Stream.of("a","b","c")
  .collect(StringBuilder::new,      // supplier：入れ物を作る
           StringBuilder::append,   // accumulator：要素を入れる
           StringBuilder::append);  // combiner：入れ物どうし
// abc
List<Integer> r = Stream.of(1,2,3)
  .collect(ArrayList::new, ArrayList::add, ArrayList::addAll);
// [1, 2, 3]
```

引数	型
supplier	Supplier<R>
accumulator	BiConsumer<R, ? super T>
combiner	BiConsumer<R, R>

accumulator と combiner は値を返さない（BiConsumer）。ArrayList::add は boolean を返すが、戻り値を捨てて使える。

reduce で List を作ると毎回新しい List が要る。入れ物にためるなら collect。

出典：API Stream.collect(Supplier, BiConsumer, BiConsumer)、java.util.stream パッケージの説明「Mutable reduction」／JDK 17 での実行結果

第2章

集める Collectors

コレクション・Map・文字列・数。

toMap の 4 つの形

```
Stream.of("a","bb","cc").collect(toMap(String::length, s -> s));
// IllegalStateException:
//   Duplicate key 2 (attempted merging values bb and cc)
Stream.of("a","bb","cc")
  .collect(toMap(String::length, s -> s, (x, y) -> x + "+" + y));
// {1=a, 2=bb+cc}（x が先の値）
Stream.of("b","a","b")
  .collect(toMap(s -> s, s -> 1, Integer::sum, TreeMap::new));
// {a=1, b=2}
Stream.of("a","b").collect(toMap(s -> s, s -> null));
// NullPointerException
```

引数	役割
1 つ目	キーを作る関数
2 つ目	値を作る関数
3 つ目	キーが重複したときのマージ（BinaryOperator）
4 つ目	Map の Supplier（TreeMap::new）

> 重複で例外になるのはキー。値が同じなのは問題ない。

出典：API Collectors.toMap（4 つの版）／JDK 17 での実行結果

コレクションと文字列に集める

メソッド	結果	注意
toList()・toSet()	List・Set	実装・変更可否は保証なし
toUnmodifiableList()	変更不可の List	null で NPE（Java 10〜）
toCollection(TreeSet::new)	TreeSet	重複なし・並ぶ
collectingAndThen(toList(), Collections::unmodifiableList)	変更不可の List	集めたあと finisher

```
Stream.of("a","b","c").collect(joining(",", "[", "]"))
// [a,b,c]
Stream.<String>empty().collect(joining(",", "[", "]"))
// []（0 個でも前後は付く）
Stream.of("a", null).collect(toList())             // [a, null]
Stream.of("a", null).collect(toUnmodifiableList()) // NPE
```

出典：API Collectors.toList・toSet・toUnmodifiableList・toCollection・collectingAndThen・joining／JDK 17 での実行結果

数を求める Collectors

メソッド	結果の型	1, 2, 4 の結果
counting()	Long	3
summingInt	Integer	7
averagingInt	Double	2.3333333333333335
summarizingInt	IntSummaryStatistics	count=3, sum=7, min=1, max=4
maxBy・minBy	Optional<T>	Optional[4]・Optional[1]

counting は Long。Map<K, Integer> には入らない。

teeing（Java 12〜）：2 つに分けて合わせる

```
Stream.of(1,2,3,4).collect(teeing(
    summingInt(n -> n),     // 10
    counting(),             // 4
    (s, c) -> (double) s / c))
// 2.5
```

IntSummaryStatistics：getCount・getSum は long、getAverage は double、getMin・getMax は int。

出典：API Collectors.counting・summingInt・averagingInt・summarizingInt・maxBy・minBy・teeing（@since 12）、IntSummaryStatistics／JDK 17 での実行結果

第3章

グループ化と下流の Collectors

groupingBy・partitioningBy と、グループごとのまとめ方。

groupingBy と partitioningBy

```
Stream.of("a","bb","cc","d").collect(groupingBy(String::length))
// {1=[a, d], 2=[bb, cc]}
Stream.of("a","bb","cc","d","eee")
  .collect(groupingBy(String::length, TreeMap::new, counting()))
// {1=2, 2=2, 3=1}（TreeMap）
Stream.of(2,4,6).collect(partitioningBy(n -> n > 10))
// {false=[2, 4, 6], true=[]}
Stream.of(1,2,3,4,5).collect(partitioningBy(n -> n % 2 == 0, counting()))
// {false=3, true=2}
```

	groupingBy	partitioningBy
キー	分類関数の結果	Boolean
要素がないキー	入らない	必ずある（空の List）
Map の Supplier	3 引数版で指定	指定できない
下流なしの値	List<T>	

> `Map<Integer, Integer> m = …groupingBy(n -> n % 2)` はコンパイルエラー（値は List）。

出典：API Collectors.groupingBy（3 つの版）・partitioningBy（2 つの版）／JDK 17 でのコンパイルと実行の結果

下流の Collectors

下流	例	結果
mapping	groupingBy(String::length, TreeMap::new, mapping(s -> s.charAt(0), toCollection(TreeSet::new))) "ab","cd","e"	{1=[e], 2=[a, c]}
filtering（Java 9〜）	groupingBy(String::length, filtering(s -> !s.startsWith("b"), toList())) "a","bb","cc","ddd"	{1=[a], 2=[cc], 3=[ddd]}
flatMapping（Java 9〜）	groupingBy(List::size, flatMapping(List::stream, toList())) [1,2], [3]	{1=[3], 2=[1, 2]}
joining	groupingBy(s -> s.charAt(0), joining("/")) "apple","avocado","banana"	{a=apple/avocado, b=banana}
reducing	groupingBy(String::length, reducing((p, q) -> p + q)) "ab","x","cd"	{1=Optional[x], 2=Optional[abcd]}
入れ子	partitioningBy(s -> s.length() > 1, groupingBy(String::length, counting())) "a","bb","cc","ddd"	{false={1=1}, true={2=2, 3=1}}

> filtering は、全部外れたグループもキーとして残る（先に filter すると残らない）。

出典：API Collectors.mapping・filtering・flatMapping・joining・reducing・groupingBy・partitioningBy／JDK 17 での実行結果

本番での問われ方

ねらい	見るところ
reduce の結果	単位元も計算に入る、単位元なしは Optional、0 個なら単位元か empty
toMap	キー重複で IllegalStateException、マージ関数の引数の順、値 null で NPE
結果の型	counting は Long、averagingInt は Double、maxBy は Optional、下流なし groupingBy は List
partitioningBy	true・false が必ずある、表示は false が先
変更できるか	toUnmodifiableList・collectingAndThen(…, unmodifiableList) は add で例外
文字列	joining の前後は全体に 1 回、0 個でも付く

出典：試験内容チェックリスト 3／本資料の問題演習（全 30 問、JDK 17 で確認）